Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware

ACT：核心方法、数学原理与训练推理算法

Action Chunking with Transformers

通过动作分块与条件变分自编码器学习精细机器人操作

作者：Yang kunpeng

论文链接：arXiv:2304.13705

代码仓库：github.com/tonyzhaozh/act

目录

第一章　核心方案：ALOHA 与 ACT **1**

第二章　核心网络结构、训练与推理算法 **3**

第一章　核心方案：ALOHA 与 ACT

精细机器人操作的难点不只是到达某个位置，而是在接触、遮挡和物体状态变化中持续进行高频闭环修正。例如，打开调味杯盖、将电池插入狭窄槽位，都需要双臂协调、精细抓取以及对视觉反馈的快速响应。仅用单步行为克隆学习映射 $o_t \mapsto a_t$ 时，某一步的小误差可能使机器人进入训练数据中很少出现的状态，之后的预测误差进一步扩大，形成误差累积（compounding errors）。此外，同一观测下，人类也可能采用多条合理轨迹；直接回归单一动作容易把多种行为平均成一条不够准确的轨迹。

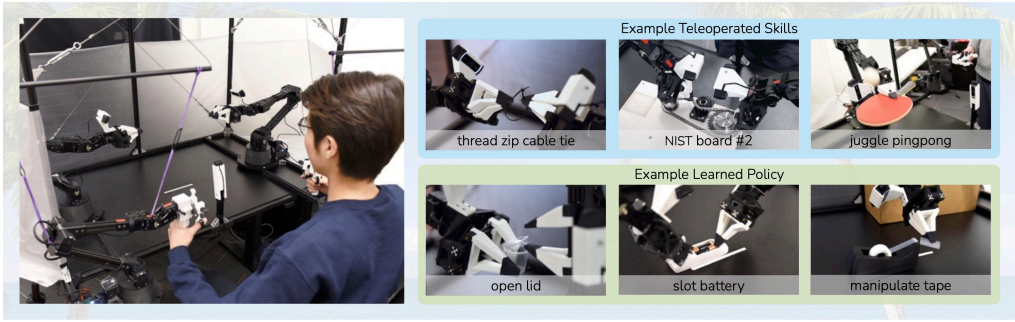


图 1.1: 原论文 Fig. 1：ALOHA 的双臂遥操作系统及其可完成的精细操作任务。操作者通过驱动主臂产生示范，跟随臂同步执行；右侧展示了遥操作技能与学习策略的任务示例。

方法从硬件与学习算法两方面解决上述问题。ALOHA（A Low-cost Open-source Hardware System for Bimanual Teleoperation）使用两台主臂和两台从臂构成双臂遥操作系统。操作者直接回驱主臂，主臂关节运动映射到从臂关节目标位置，从而以较低延迟采集精细操作示范。系统使用四路 RGB 相机：前视、顶视以及左右腕部相机；图像分辨率为 480×640，遥操作与数据记录频率为 50 Hz。主臂关节位置被记录为动作标签，从臂当前关节位置与多视角图像构成策略观测。双臂各有 6 个机械臂自由度和 1 个夹爪维度，因此动作向量为 14 维。低层电机 PID 控制器跟踪策略给出的关节目标。



图 1.2: 原论文 Fig. 3：ALOHA 的相机视角、双臂工作空间、定制夹爪机构及 ViperX 从臂规格。多视角图像为策略提供互补的环境信息，腕部相机尤其适合观察夹爪与操作对象之间的局部关系。

算法层面的核心是将“每一步预测一个动作”改为“每次预测未来一段动作”。令当前观测为

$$o_t = (I_t^1, I_t^2, I_t^3, I_t^4, q_t),$$

其中 I_t^c 表示第 c 路相机图像，q_t 表示机器人当前关节位置。长度为 k 的动作块记为

$$A_t = (a_t, a_{t+1}, \ldots, a_{t+k-1}) \in \mathbb{R}^{k\times 14}.$$

ACT 学习条件策略 $\pi_\theta(A_t \mid o_t)$，根据当前观测一次生成未来 k 步的关节目标。动作分块把长轨迹划分为较短的预测单元，使模型更容易学习连续动作的局部结构，并减少逐步预测时误差反复传递的影响。常见的 ALOHA 配置使用 $k = 100$，但动作块长度应结合控制频率与任务特点选择。它也能表达示范中的时间相关行为，例如操作中短暂停顿：同一个瞬时状态不一定足以确定下一动作，动作块可同时学习该状态附近的一段时间结构。

动作分块并不意味着机器人必须开环执行完整动作块。若每隔 k 步才重新观察一次，新的观测会突然改变动作计划，可能造成运动不连续。因此，ACT 还采用时间集成（temporal ensembling）：每个控制时刻重新预测未来动作块，使不同预测块在时间轴上重叠；对同一个执行时刻收到的多个候选动作做指数加权平均，再执行融合后的当前动作。动作分块负责预测一段连贯的动作，时间集成负责协调不同观测时刻产生的重叠预测。

ACT 用 CVAE 表示人类示范中的多模态性：训练时，Encoder 根据真实动作块与当前关节状态推断 z，策略再根据图像、关节状态和 z 预测动作块。KL 项约束近似后验接近标准正态先验；推理时没有真实动作块，故移除 Encoder 并设 $z = 0$，使策略根据当前观测确定性地生成动作。

第二章　核心网络结构、训练与推理算法

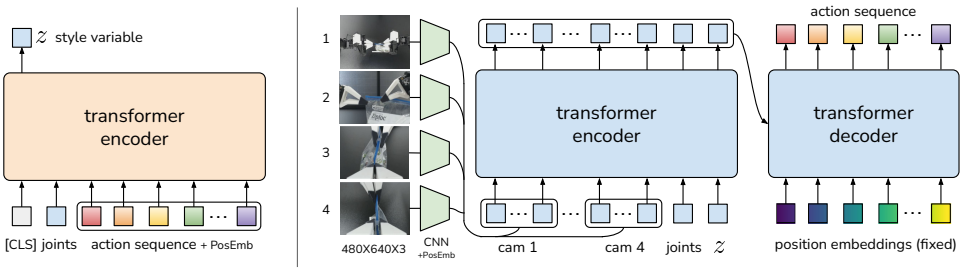


图 2.1: 原论文 Fig. 4：ACT 的 CVAE 与 Transformer 网络结构。左侧是训练阶段使用的 CVAE Encoder；右侧是执行时保留的策略网络。

图 2.1 中有两个容易混淆的 Encoder。左侧的 CVAE Encoder 只在训练阶段使用，它接收真实动作块和当前关节状态，不接收图像；右侧策略网络内部的 Transformer Encoder 则融合视觉特征、关节状态特征和潜变量 z。二者任务不同，不能因为都叫 Encoder 而视为同一个模块。

设真实动作块为 A_t，当前关节状态为 $\bar{o}_t = q_t$。CVAE Encoder 参数化近似后验分布

$$q_\phi(z \mid A_t, \bar{o}_t) = \mathcal{N}\big(\mu_\phi, \mathrm{diag}(\sigma_\phi^2)\big)\,.$$

实现中将关节位置与动作块分别通过线性层映射到隐藏维度，再与可学习的 [CLS] token 拼接，送入类似 BERT 的 Transformer Encoder。取 [CLS] 对应的输出，经线性层预测 μ_ϕ 和 $\log \sigma_\phi^2$。潜变量维度为 32。由于采样操作本身不可直接进行普通反向传播，模型使用重参数化技巧：

$$z = \mu_\phi + \sigma_\phi \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad \sigma_\phi = \exp\big(\tfrac{1}{2}\log \sigma_\phi^2\big)\,.$$

这样，随机性由独立噪声 ϵ 表示，梯度仍可沿 μ_ϕ 与 σ_ϕ 传回 Encoder。

策略网络接收当前多视角图像、关节位置和 z。每路图像先经过 ResNet-18 提取特征图，再展平为视觉 token，并加入二维正弦位置编码以保留空间位置信息；不同相机的 token 随后拼接。关节位置与潜变量分别经过线性投影后加入序列，策略内部的 Transformer Encoder 融合这些信息。Transformer Decoder 使用固定的位置查询嵌入生成未来 k 个动作，动作头将每个查询位置的隐藏特征映射为 14 维关节目标。策略输出为

$$\hat{A}_t = \pi_\theta(o_t, z) \in \mathbb{R}^{k\times 14}.$$

因此，潜变量是条件输入，动作块才是模型预测的监督目标和实际控制量。

Algorithm 1 ACT Training

Given: Demo dataset $\mathcal{D}$, chunk size k, weight β.
Let a_t, o_t represent action and observation at timestep t, $\bar{o}_t$ represent o_t without image observations.
Initialize encoder $q_\phi(z \mid a_{t:t+k}, \bar{o}_t)$
Initialize decoder $\pi_\theta(\hat{a}_{t:t+k} \mid o_t, z)$
for iteration $n = 1, 2, \ldots$ **do**
    Sample o_t, $a_{t:t+k}$ from $\mathcal{D}$
    Sample z from $q_\phi(z \mid a_{t:t+k}, \bar{o}_t)$
    Predict $\hat{a}_{t:t+k}$ from $\pi_\theta(\hat{a}_{t:t+k} \mid o_t, z)$
    $\mathcal{L}_{reconst} = MSE(\hat{a}_{t:t+k}, a_{t:t+k})$
    $\mathcal{L}_{reg} = D_{KL}\left(q_\phi(z \mid a_{t:t+k}, \bar{o}_t) \,\|\, \mathcal{N}(0, I)\right)$
    Update θ, ϕ with ADAM and $\mathcal{L} = \mathcal{L}_{reconst} + \beta\mathcal{L}_{reg}$
end for

训练目标由动作重建损失与 KL 正则项组成。论文 Algorithm 1 将重建损失写为均方误差（MSE）：

$$\mathcal{L}_{\text{MSE}} = \frac{1}{k d_a} \sum_{j=0}^{k-1} \sum_{d=1}^{d_a} \left(\hat{a}_{t+j,d} - a_{t+j,d}\right)^2, \qquad d_a = 14.$$

它对每个未来时间步、每个关节的预测误差平方后求平均；较大的误差会受到更强惩罚。该项将梯度传给策略网络，并通过重参数化路径传回 CVAE Encoder，使潜变量保留有助于重建演示动作的信息。

需要区分的是，官方 ACT 代码中 `ACTPolicy` 的实际重建项不是 MSE，而是带 padding mask 的 L1 损失。动作序列在 episode 末尾可能通过 padding 补齐到固定长度，`is_pad` 标记无效位置，避免它们对重建误差产生贡献。实现的核心计算如下：

```
all_l1 = F.l1_loss(actions, a_hat, reduction='none')
l1 = (all_l1 * ~is_pad.unsqueeze(-1)).mean()

loss_dict['l1'] = l1
loss_dict['kl'] = total_kld[0]
loss_dict['loss'] = loss_dict['l1'] + loss_dict['kl'] * self.
    kl_weight
```

L1 损失计算绝对误差，而非平方误差。这里保留了论文算法伪代码中的 MSE 公式，同时明确说明实际 ACTPolicy 使用 masked L1；不能把伪代码和实现混为一谈。代码中的 `reduction='none'` 先保留每个元素的误差，再通过 `~is_pad.unsqueeze(-1)` 将 padding 位置置零，最后求均值。

KL 正则项约束 Encoder 输出的近似后验接近固定先验 $p(z) = \mathcal{N}(0, I)$。对于一个 d_z 维、协方差为对角矩阵的高斯分布，有

$$\begin{aligned}\mathcal{L}_{\mathrm{KL}} &= D_{\mathrm{KL}}(q_\phi \| p) \\ &= \frac{1}{2} \sum_{j=1}^{d_z} \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\right).\end{aligned}$$

其中 μ_j 与 σ_j^2 分别是潜变量第 j 维的均值和方差；当 $\mu_j = 0$、$\sigma_j^2 = 1$ 时，该维度的 KL 贡献为零。KL 并不是要求每次采样出的 z 都等于零，而是约束概率分布的形状与位置。$p(z)$ 是固定目标，训练真正调整的是由参数 ϕ 决定的 q_ϕ。

代码直接预测 mu 与 `logvar`，其中 `logvar`，即 $\log \sigma^2$。对角高斯相对于标准正态先验的 KL 可以逐维计算为

$$\ell_j = -\frac{1}{2}\left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right),$$

再对潜变量维度求和、对 batch 求平均。对应实现为：

```
klds = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp())
total_kld = klds.sum(1).mean(0, True)
```

这里 `logvar.exp()` 就是方差 σ^2。最终损失为 $\mathcal{L} = \mathcal{L}_{\text{reconst}} + \beta \mathcal{L}_{\mathrm{KL}}$。重建项推动预测动作贴近示范；KL 项限制潜变量分布不要过度偏离先验。β 越大，正则约束越强，潜变量可携带的演示特定信息通常越少。两项损失共同反向传播：策略参数 θ 通过动作重建项更新，Encoder 参数 ϕ 同时受到重建梯度与 KL 梯度影响。

推理阶段不再提供真实动作块，因此无法用 $q_\phi(z \mid A_t, \bar{o}_t)$ 推断潜变量。标准正态先验的均值为零，故 ACT 移除 CVAE Encoder，直接设 $z = 0$，并计算 $\hat{A}_t = \pi_\theta(o_t, 0)$。这使策略在给定观测时确定性地输出动作块。零向量不是一个动作，也不意味着训练时 Encoder 被要求把所有样本编码成零；它是推理阶段采用的固定先验输入。

Algorithm 2 ACT Inference

1: Given: trained π_θ, episode length T, weight m.
2: Initialize FIFO buffers $\mathcal{B}[0:T]$, where $\mathcal{B}[t]$ stores actions predicted *for* timestep t.
3: **for** timestep $t = 1, 2, \ldots, T$ **do**
4:　　Predict $\hat{a}_{t:t+k}$ with $\pi_\theta(\hat{a}_{t:t+k} \mid o_t, z)$ where $z = 0$
5:　　Add $\hat{a}_{t:t+k}$ to buffers $\mathcal{B}[t:t+k]$ respectively
6:　　Obtain current step actions $A_t = \mathcal{B}[t]$
7:　　Apply $a_t = \sum_i w_i A_t[i] / \sum_i w_i$, with $w_i = \exp(-m * i)$
8: **end for**

缓冲区 $\mathcal{B}$ 的索引对应的是动作的目标执行时刻，而不是预测产生的时刻。模型在时刻 t 预测未来 k 步，就把第 j 个预测动作放入 $\mathcal{B}[t+j]$。例如，$\mathcal{B}[t+2]$ 可以同时保存较早时刻和较新时刻对执行时刻 $t+2$ 的预测。当前要执行时刻 t 时，只读取 $\mathcal{B}[t]$，把这些候选动作融合为一个动作。这样聚合的是“不同观测时刻对同一个执行时刻的预测”，而不是将相邻执行时刻的动作混合。

时间集成使用指数权重

$$w_i = \exp(-mi), \qquad a_t = \frac{\sum_i w_i A_t[i]}{\sum_i w_i}.$$

按算法的索引约定，$i = 0$ 对应最早的预测，因此它的原始权重为 1；后续预测的权重随 i 增大而指数衰减。m 是人为设定的衰减系数，不是训练得到的网络参数。m 较小时，各预测的权重更接近，新观测产生的预测能够更快影响执行动作；m 较大时，权重更偏向旧预测，吸收新观测的速度更慢。时间集成不增加训练损失，只增加推理时保存和融合预测的计算。

在官方推理实现中，时间集成开启时将查询频率设为每个控制时刻一次，并把预测写入三维张量 `all_time_actions`。概念上它等价于论文中的 $\mathcal{B}$：第一维记录预测产生的时刻，第二维记录目标执行时刻，第三维是动作向量。核心计算如下：

```
all_time_actions[[t], t:t+num_queries] = all_actions
actions_for_curr_step = all_time_actions[:, t]
```

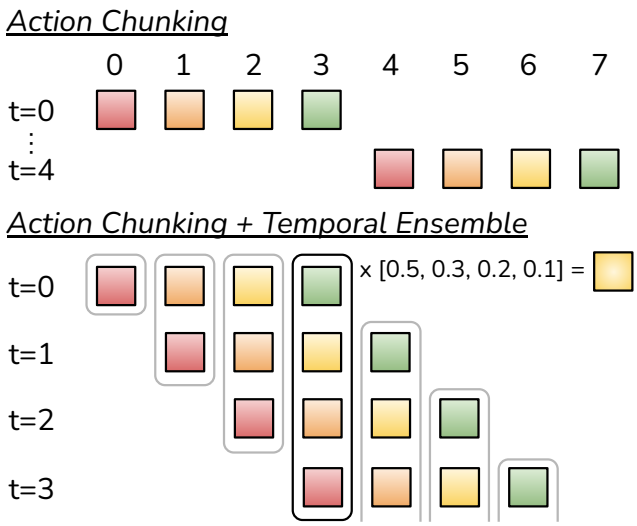


图 2.2: 原论文 Fig. 5：动作分块与时间集成。每次查询都会产生一个覆盖未来多个时刻的动作块；重叠位置的多个预测通过加权平均形成实际执行动作。

```
actions_for_curr_step = actions_for_curr_step[actions_populated]

k = 0.01  # exponential decay coefficient, not chunk length
exp_weights = np.exp(-k * np.arange(len(actions_for_curr_step)))
exp_weights = exp_weights / exp_weights.sum()
raw_action = (actions_for_curr_step * exp_weights[:, None]).sum(
    dim=0)
```

这里代码局部变量名 k 被用于指数衰减系数，容易与动作块长度混淆；它的数值 ‘0.01‘ 对应算法中的 m，而动作块长度由 num_queries 表示。若关闭时间集成，策略按动作块频率查询并依次执行块内动作；若开启，则每步重新查询并融合当前执行时刻的重叠预测。最终动作经数据反归一化后交给低层控制器跟踪，环境返回新的图像与关节状态，系统进入下一轮闭环推理。